Software Requirmnets Specification

For

# Tour -Visit Planner (DayTripGO)

*A Web-Based Visit Planning System for Wadduwa and Surrounding Areas*

Document Version: 1.0

Student Name - De Silva P.K.M
Registration Number - E2410884
Date: 21.08.2026

# Revision History

| Version | Date | Author | Description of Change |
|---|---|---|---|
| 1.0 | 21.08.2026 | P.K.M.DE Silva | Initial draft of SRS based on approved project proposal. |

# Table of Contents

Revision History ..... 2
Table of Contents ..... 3
1.1 Purpose ..... 4
1.2 Document Conventions ..... 4
1.3 Intended Audience and Reading Suggestions ..... 4
1.4 Project Scope ..... 4
1.5 References ..... 5
2. Overall Description ..... 5
2.1 Product Perspective ..... 5
2.2 Product Functions ..... 5
2.3 User Classes and Characteristics ..... 6
2.4 Operating Environment ..... 6
2.5 Design and Implementation Constraints ..... 6
2.6 User Documentation ..... 7
2.7 Assumptions and Dependencies ..... 7
3. External Interface Requirements ..... 7
3.1 User Interfaces ..... 7
3.2 Hardware Interfaces ..... 8
3.3 Software Interfaces ..... 8
3.4 Communications Interfaces ..... 8
4. Functional Requirements ..... 9
4.1 Use Case Summary ..... 10
5. Non-Functional Requirements ..... 11
5.1 Safety Requirements ..... 12
5.2 Security Requirements ..... 12
5.3 Software Quality Attributes ..... 12
6. Data Requirements ..... 12
6.1 Place Entity ..... 13
6.2 Category Entity ..... 13
6.3 VisitPlan and VisitPlanItem Entities ..... 13
6.4 Administrator Entity ..... 13
7. Requirements Traceability ..... 14
8. Appendices ..... 15
Appendix A: Glossary ..... 15
Appendix B: Open Issues ..... 15
B.1 Sequence Diagrams ..... 15
B.2 Class Diagram ..... 16
B.3 ER Diagram ..... 17
Appendix C: Open Issues ..... 17

# 1. Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Local Tourist Day-Visit Planner and Information System, a web-based application that helps tourists discover places of interest within 25 km of Wadduwa, Sri Lanka, and build a simple one day visit plan. This document is intended for the developer, the module supervisor/assessor, and the four consulted stakeholders, and it forms the basis for design, implementation and testing of the system under module ITE2953.

## 1.2 Document Conventions

Requirements are labelled using the prefixes FR (Functional Requirement) and NFR (Non-Functional Requirement), each followed by a sequential number, e.g. FR-01. The keywords "shall" and "must" indicate mandatory requirements; "should" indicates a desirable but non-mandatory requirement. Priority is expressed as High, Medium or Low, where High requirements are necessary for the system to fulfil its core purpose.

## 1.3 Intended Audience and Reading Suggestions

- Developer: primary audience; uses Sections 2–6 for design and implementation guidance.
- Module Supervisor / Assessor: uses Sections 1–3 to verify the requirements match the approved proposal, and Section 4 to trace testable requirements.
- Stakeholders (tourist representatives): may review Section 2 (Overall Description) and Section 4 (Functional Requirements) in plain language to confirm their needs have been captured correctly.

## 1.4 Project Scope

The system scope, as approved in the project proposal, covers: browsing and filtering at least 10 verified places of interest within 25 km of Wadduwa; viewing place details including description, opening times, travel tips and distance; viewing each place on a map; building and viewing a simple one-day visit plan; and an administrator interface for adding, editing and deleting place records. Booking, payment, multi-day itineraries and transport/accommodation reservations are explicitly out of scope.

### 1.5 References

[1] Project Proposal: "Local Tourist Day-Visit Planner and Information System," ITE2953 Programming Group Project 25S1.

[2] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications (used as the structural template for this document).

## 2. Overall Description

### 2.1 Product Perspective

The Tourist Day-Visit Planner is a new, standalone web application; it is not a component of, or replacement for, an existing system. It consists of a public tourist-facing interface, a restricted administrator interface, and a shared relational database of places, categories and visit plans. The system depends on a third-party map-embedding service (e.g. Google Maps or OpenStreetMap) to plot place locations, and this is treated as an external interface rather than something built from scratch (see Section 5).

### 2.2 Product Functions

At a high level, the system provides the following functions, summarised here and detailed as individual requirements in Section 4:

- List and filter places of interest by category (Tourist).
- View detailed information for a single place, including a map of its location (Tourist).
- Add places to, remove places from, and view a one-day visit plan (Tourist).
- Authenticate as an administrator (Administrator).
- Create, update and delete place records (Administrator).

### 2.3 User Classes and Characteristics

| User Class | Description | Technical Expertise |
|---|---|---|
| Tourist | Domestic or foreign visitor staying in or near Wadduwa who browses places of interest and builds a one-day visit plan. Primary, unauthenticated user. | Low to moderate; assumed familiar with basic web browsing on desktop or mobile. |
| Administrator | Person responsible for keeping place-of-interest data accurate; logs in through a restricted interface to add, edit or delete records. | Moderate; comfortable with web forms, no coding knowledge required. |

### 2.4 Operating Environment

- Client: modern desktop and mobile web browsers (Chrome, Firefox, Edge, Safari, latest two major versions).
- Server: Node.js runtime with an Express-based backend (or the module-approved equivalent).
- Database: MySQL relational database.
- Hosting: development/demo environment suitable for local or low-cost cloud hosting for the duration of the module; production-grade hosting is out of scope.

### 2.5 Design and Implementation Constraints

- The technology stack must match the stack approved for ITE2953 (see Section 1.4 of the proposal); any deviation requires supervisor approval.
- The system must be delivered without payment, booking or reservation functionality, per the approved scope.
- The place dataset must contain a minimum of 10 verified places within 25 km of Wadduwa at the point of submission.

- Development is carried out by a single student within the timeframe of one academic module, which constrains the depth of testing and the size of the feature set.

### 2.6 User Documentation

A short user guide (or in-app help text) covering how to filter places, view details, build a one-day plan, and how an administrator logs in and manages records will be delivered alongside the system, in line with standard ITE2953 submission requirements.

### 2.7 Assumptions and Dependencies

- It is assumed that opening times and travel tips supplied by place owners/authorities do not change frequently enough to require real-time synchronisation.
- The system depends on an external map-embedding API remaining available and free-to-use at the tier required for a student project.
- It is assumed that the four consulted stakeholders are broadly representative of the primary Tourist user class for the purpose of requirements validation.

## 3. External Interface Requirements

### 3.1 User Interfaces

- Home, Place Listing page - displays places of interest as a filterable list or grid, with category filter controls.
- Place Detail page - displays a single place's description, opening times, travel tips, distance from Wadduwa, and an embedded map.
- Visit Plan page - displays the tourist's currently selected places for a one-day plan, in a single consolidated view.
- Administrator Login page - a restricted form for administrator authentication.
- Administrator Dashboard - a form based interface for adding, editing and deleting place records.

Detailed wireframes/mockups are to be produced in Figma during the design phase and referenced here as figures once available (see List of Figures).

### 3.2 Hardware Interfaces

No dedicated hardware is required. The system relies on the client device's existing display, input and network hardware; no sensors, printers or other peripherals are used.

### 3.3 Software Interfaces

| Interface | Type | Purpose |
|---|---|---|
| Map Service API | Third-party web API | Renders an embedded map and marker for each place's location on the Place Detail page. |
| MySQL Database | Relational database | Stores place, category, visit-plan and administrator account data; accessed by the backend only. |
| Backend REST API | Internal API | Exposes endpoints consumed by the frontend for listing, filtering, detail retrieval, visit-plan management and administrator CRUD operations. |

### 3.4 Communications Interfaces

All communication between the client browser and the backend occurs over HTTPS using standard HTTP request/response semantics (REST). No email, messaging, or real-time push protocols are required by the current scope.

## 4. Functional Requirements

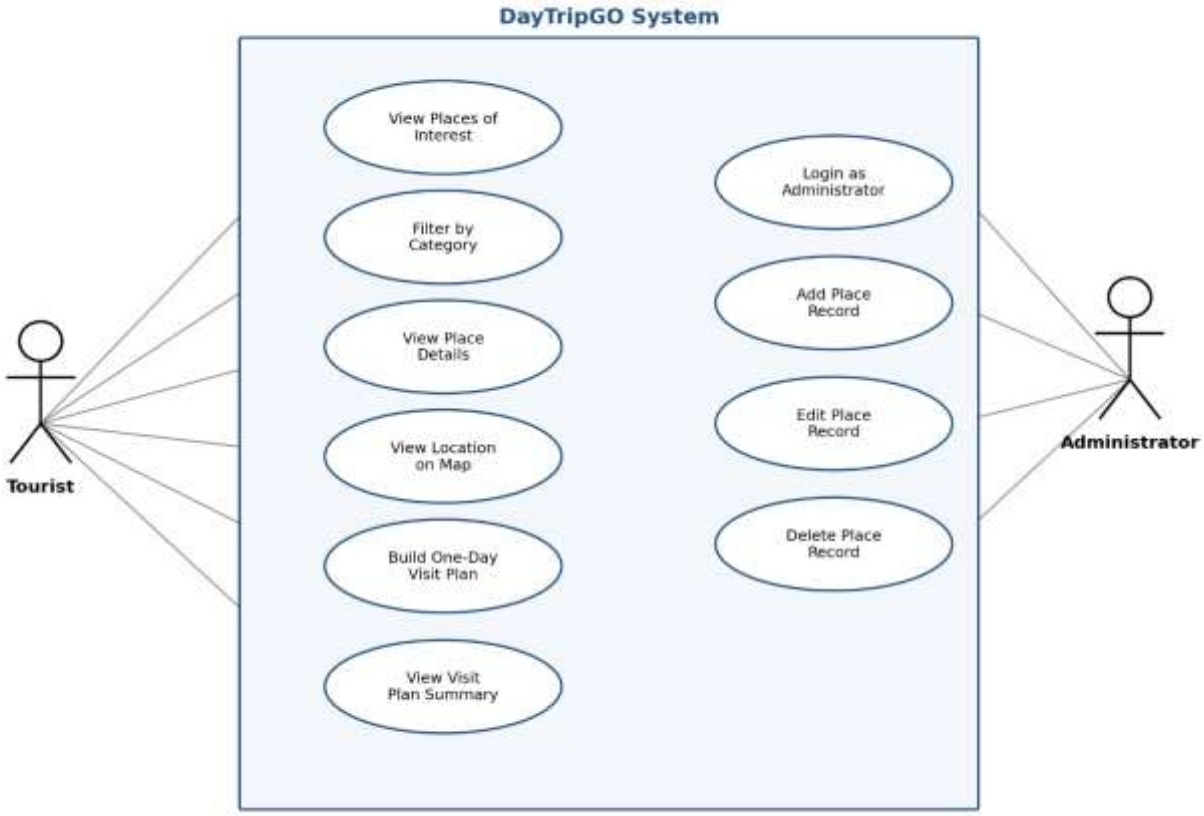
This section lists the functional requirements traced from the objectives and system overview in the approved project proposal. Each requirement is independently testable and mapped to a primary actor.

| ID | Requirement Description | Actor | Priority |
|---|---|---|---|
| FR-01 | The system shall display a list of all places of interest on the home page. | Tourist | High |
| FR-02 | The system shall allow the tourist to filter the place list by category (Nature, Religious, Heritage, Cultural). | Tourist | High |
| FR-03 | The system shall allow the tourist to select a place from the list to view its detail page. | Tourist | High |
| FR-04 | The system shall display, for each place, its description, opening times, travel tips and approximate distance from Wadduwa. | Tourist | High |
| FR-05 | The system shall display each place's location on an embedded map on its detail page. | Tourist | High |
| FR-06 | The system shall allow the tourist to add a place to a one-day visit plan. | Tourist | High |
| FR-07 | The system shall allow the tourist to remove a previously added place from the visit plan. | Tourist | Medium |
| FR-08 | The system shall display the tourist's current visit plan as a single consolidated view listing all selected places. | Tourist | High |
| FR-09 | The system shall prevent booking, payment or reservation actions from being presented anywhere in the tourist interface. | Tourist | Medium |
| FR-10 | The system shall require an administrator to log in with valid credentials before accessing the administrator dashboard. | Administrator | High |
| FR-11 | The system shall allow an authenticated administrator to create a new place record with all | Administrator | High |

| | | | |
|---|---|---|---|
| | required fields (name, category, description, opening times, tips, distance, location). | | |
| FR-12 | The system shall allow an authenticated administrator to edit an existing place record. | Administrator | High |
| FR-13 | The system shall allow an authenticated administrator to delete an existing place record. | Administrator | High |
| FR-14 | The system shall validate required fields on place-creation and place-editing forms before saving. | Administrator | Medium |
| FR-15 | The system shall log an administrator out and end their session on request or after a period of inactivity. | Administrator | Low |

### 4.1 Use Case Summary

**Figure: Use Case Diagram — DayTripGO**



## 5. Non-Functional Requirements

| ID | Category | Requirement Description | Priority |
|---|---|---|---|
| **NFR-01** | Usability | A first-time tourist user shall be able to locate and view a place's details within 3 interactions from the home page, without instruction. | High |
| **NFR-02** | Performance | The place listing page shall load within 3 seconds on a standard broadband connection with the initial dataset (approx. 10–20 places). | Medium |
| **NFR-03** | Availability | The system should be available and demonstrable throughout the module's assessment period, excluding scheduled maintenance. | Medium |
| **NFR-04** | Security | Administrator credentials shall be stored using a salted hash, and the administrator dashboard shall not be accessible without a valid authenticated session. | High |
| **NFR-05** | Compatibility | The tourist interface shall render correctly on the latest two major versions of Chrome, Firefox, Edge and Safari, on both desktop and mobile viewport widths. | Medium |
| **NFR-06** | Maintainability | Place data shall be editable by an administrator through the UI without requiring changes to application code. | High |
| **NFR-07** | Data Integrity | The system shall prevent a place record from being saved with missing required fields (name, category, distance). | Medium |
| **NFR-08** | Scalability | The database schema shall support at least 100 place records without requiring structural changes. | Low |

### 5.1 Safety Requirements

The system handles no payment, medical, or safety-critical data; no dedicated safety requirements beyond standard data-handling practice apply.

### 5.2 Security Requirements

- Only authenticated administrators shall be able to create, edit or delete place records (see NFR-04).
- All administrator authentication traffic shall be transmitted over HTTPS.
- Tourist-facing browsing shall not require account creation or collect personal data beyond what is strictly needed for the visit-plan feature (which is session-based, not account-based).

### 5.3 Software Quality Attributes

- Reliability: core browsing and filtering functions (FR-01–FR-05) shall function correctly for all records in the initial dataset.
- Portability: the backend shall run on any standard Node.js hosting environment without code changes.
- Testability: every functional requirement in Section 4 shall map to at least one test case in the system test plan.

## 6. Data Requirements

The core entities are Place, Category, VisitPlan, VisitPlanItem and Administrator. The Place table, the entity most central to the system, is detailed below as an example; the remaining entities follow the same convention and will be finalised as an entity-relationship diagram during design (see List of Figures).

### 6.1 Place Entity

| Field | Type / Constraint | Description |
|---|---|---|
| **place_id** | INT, PK, auto-increment | Unique identifier for a place. |
| **name** | VARCHAR(150), NOT NULL | Name of the place of interest. |
| **category_id** | INT, FK -> category.category_id | Links the place to a category. |
| **description** | TEXT, NOT NULL | Descriptive text shown on the detail page. |
| **opening_hours** | VARCHAR(100) | Opening times, free text (e.g. "6:00 AM – 8:00 PM daily"). |
| **travel_tips** | TEXT | Practical tips for visiting. |
| **distance_km** | DECIMAL(4,1), NOT NULL | Approximate distance from Wadduwa in kilometres. |
| **latitude** | DECIMAL(9,6), NOT NULL | Latitude used for map display. |
| **longitude** | DECIMAL(9,6), NOT NULL | Longitude used for map display. |
| **created_at** | DATETIME | Record creation timestamp. |
| **updated_at** | DATETIME | Record last-updated timestamp. |

### 6.2 Category Entity

- category_id (INT, PK, auto-increment)
- category_name (VARCHAR(50), NOT NULL, UNIQUE) — e.g. Nature, Religious, Heritage, Cultural

### 6.3 VisitPlan and VisitPlanItem Entities

- VisitPlan: plan_id (PK), session_id or device token, created_at.
- VisitPlanItem: item_id (PK), plan_id (FK), place_id (FK), sort_order — links selected places to a plan in a chosen order.

### 6.4 Administrator Entity

- admin_id (PK), username (UNIQUE, NOT NULL), password_hash (NOT NULL), created_at

## 7. Requirements Traceability

This table maps each project objective from the approved proposal to the requirement(s) in this SRS that realise it, to support verification that the SRS is complete relative to the proposal.

| Project Objective | Related Requirement(s) |
|---|---|
| **Identify ≥10 verified places within 25 km of Wadduwa** | FR-01, FR-04, Section 6.1 |
| **Elicit and validate requirements through stakeholder consultation** | Section 2.3, Appendix A (proposal) |
| **Design a relational database and system architecture** | Section 2.1, Section 6 |
| **Implement viewing, filtering and mapping of places** | FR-01–FR-05 |
| **Implement a one-day visit-plan builder** | FR-06–FR-08 |
| **Implement administrator add/edit/delete functionality** | FR-10–FR-14 |

## 8. Appendices

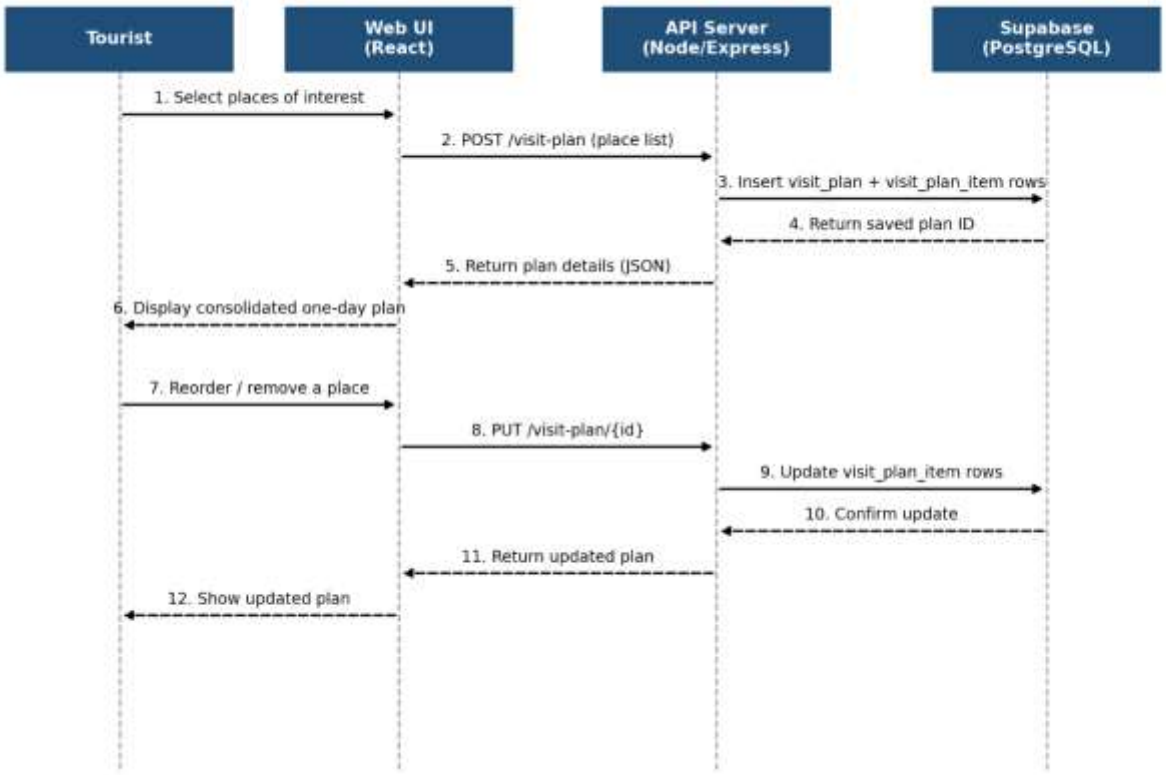
### Appendix A: Glossary

| Term | Definition |
|---|---|
| **Place of interest** | A location (religious, nature, heritage or cultural site) documented in the system for tourists to view and plan visits to. |
| **Visit plan** | A tourist's selected, ordered set of places forming a single day's itinerary within the system. |
| **Administrator** | An authenticated user role responsible for maintaining place data. |

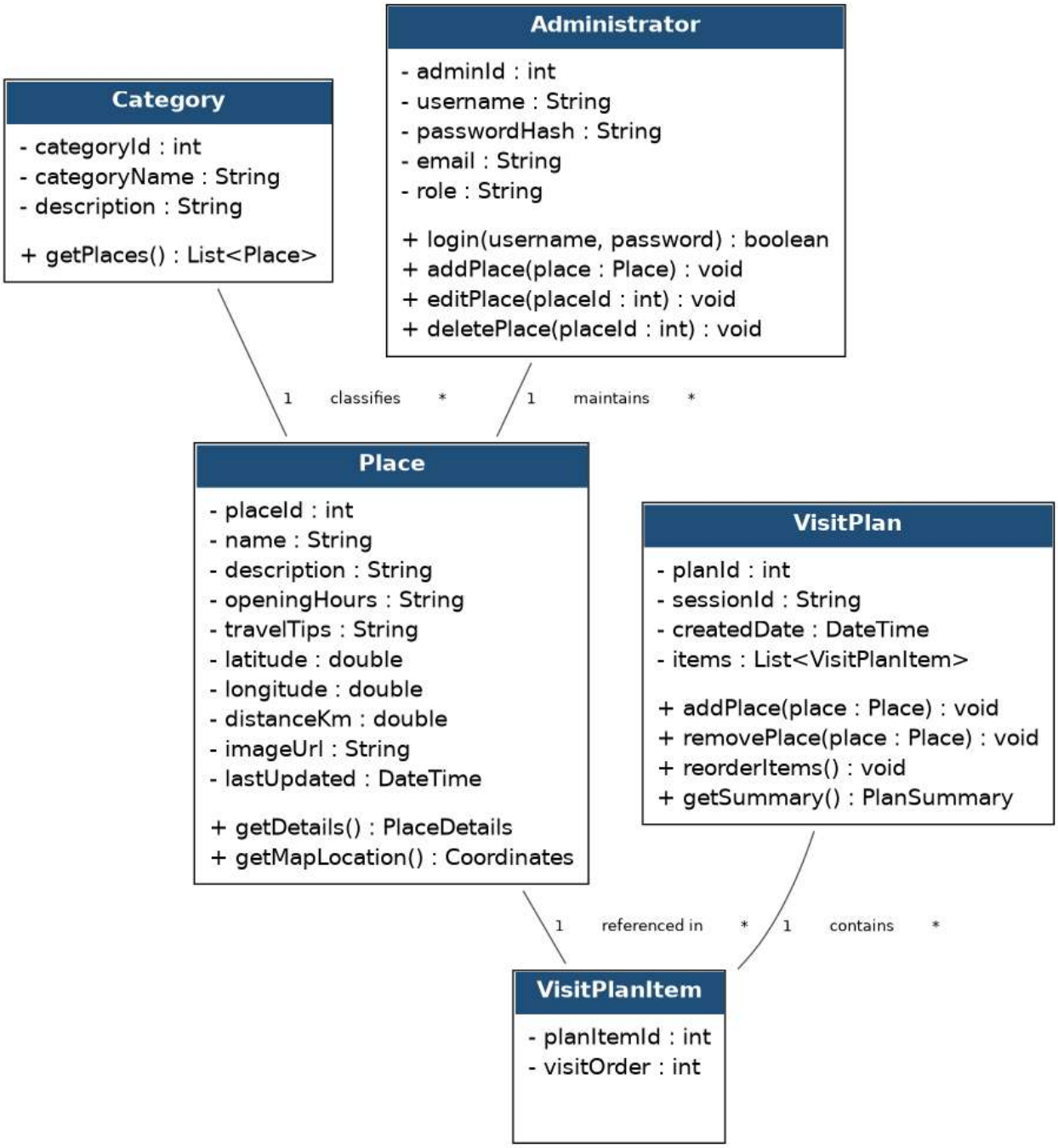
### Appendix B: Open Issues

## B.1 Sequence Diagrams

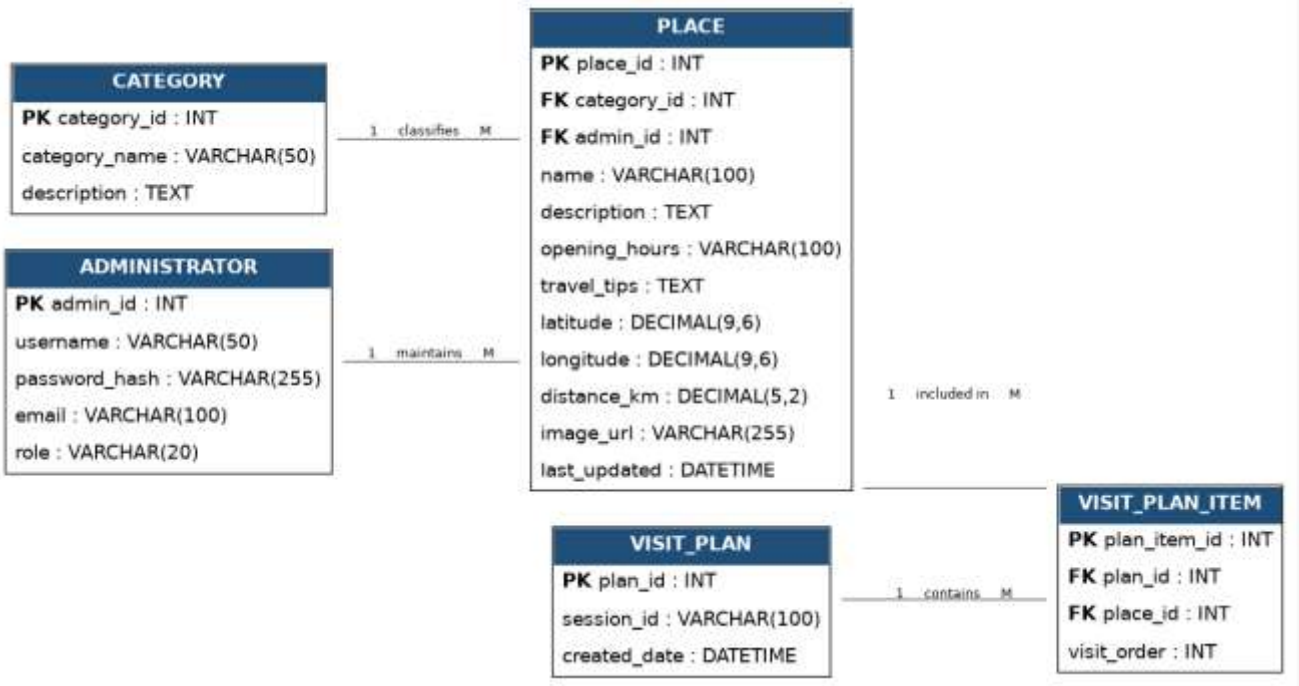
**Figure: Sequence Diagram — Build One-Day Visit Plan**



## B.2 Class Diagram



## B.3 ER Diagram




## Appendix C: Open Issues

- Final confirmation of the technology stack (Node.js/Express + MySQL vs. PHP alternative) pending module approval.
- Exact distance verification for the Kosgoda Sea Turtle Conservation Project, which sits close to the 25 km boundary.
- Selection of the specific map-embedding API (Google Maps vs. OpenStreetMap/Leaflet) pending cost/quota review for a student project.